

bash scripting

Syntax of a command in CLI

- CLI is a tool into which one can type commands to perform tasks.
- An all-text display
- prompt (~ % or ~\$): short text message at the start of command line
- Standard input > Standard output
- Linux commands are case-sensitive
- % **touch** file1.txt # **command** argument
- % **touch -c** file1.txt # **command** **parameter** (not create if exist) argument
- % **cp -n** file1 file 2 # **command** **parameter** (no overwrite) argument

Get help for the commands

man **command** = look up the documentation (don't forget the q to exit)
command # no arguments = displays a short help for the command

% **touch** file1.txt # **command** argument

% man **touch** (**man --help**)

% **touch**

usage: touch [-A [-][[hh]mm]SS] [-achm] [-r file]

Text editors

Editor	Interface	Learning curve	Extensibility / plugins	LSP / IntelliSense	Git workflow	Remote over SSH	Resource use	Best fit
nano	Terminal	Low	Minimal	No	No	Excellent (just SSH)	Very low	Quick edits, beginners, servers
Vim	Terminal	High	Very high (plugins)	Yes (via plugins)	Yes (via plugins)	Excellent	Low	Fast editing, sysadmin, power users
Neovim	Terminal	High	Very high (modern plugin)	Yes (strong LSP)	Yes (via plugins)	Excellent	Low–Med	Vim-style editing + modern IDE features
Emacs	Terminal/GUI	High	Extremely high (packages)	Yes (packages)	Yes (packages)	Excellent	Med–High	“Do everything” editor, heavy customization
micro	Terminal	Low–Med	Moderate (plugins)	Limited (via plugins)	Limited	Excellent	Low	Friendly terminal editor with modern defaults
Gedit / GNOME	GUI	Low	Low–Moderate	Limited	Limited	N/A (use SSHFS/SFTP separately)	Low	Simple GUI editing, notes, small scripts
Kate	GUI	Low–Med	Moderate–High	Some (plugins)	Some (plugins)	N/A (use SSHFS/SFTP separately)	Low–Med	Developer-friendly GUI editor (KDE), projects
VS Code	GUI	Low–Med	Very high (extensions)	Yes (strong)	Yes (strong)	Yes (Remote SSH/Containers)	Med–High	Most people’s “IDE-lite” for coding
Sublime Text	GUI	Low–Med	High	Yes (via packages)	Yes (via packages)	Limited (usually via plugins)	Low–Med	Fast GUI editor, big files, keyboard-centric
JetBrains IDEs (PyCharm/IntelliJ/etc.)	GUI	Med	High (plugins)	Yes (excellent)	Yes (excellent)	Yes (remote interpreters/SSH options)	High	Full IDE features, refactoring, large codebases

On servers / minimal environment: nano (easy) or vim/neovim (power).

- Nano filename.txt
 - **Save:** Ctrl + O (Write Out)
 - **Exit nano:** Ctrl + X
 - Search: Ctrl + W
 - Cut line: Ctrl + K
 - Paste line: Ctrl + U
 - Help: Ctrl + G

- Vim

:w	# Save (write) the current file
:w newname.txt	# Save as a new name
:wq	# Save and quit
:q	# Quit (only works if no unsaved changes)

- VS Code

code .	# Open a folder (current directory) in VS Code:
code original.txt	# Open one or more file

VS Code

- `sudo apt install -y wget gpg`
- `wget -qO- https://packages.microsoft.com/keys/microsoft.asc | gpg --dearmor | sudo tee /usr/share/keyrings/microsoft.gpg > /dev/null`
- `echo "deb [arch=amd64 signed-by=/usr/share/keyrings/microsoft.gpg] https://packages.microsoft.com/repos/code stable main" | sudo tee /etc/apt/sources.list.d/vscode.list`
- `sudo snap install code --classic`

root (OS) command vs shell environment variable & paths

- `echo $SHELL` # Which shell am I using?
- `whoami = echo $USER` # root (OS) shell environment variable
- `pwd = echo $PWD`
- `echo $PWD = echo $HOME` # when your current directory is your home
- `$HOME`, `$USER` and `$PWD` are shell variables
- `/Users/mohma/CQ/test/test2/file_test2.txt` # absolute path
- `cat ./test2/file_test2.txt` # relative path (while inside `/Users/mohma/CQ/test`)
- `cat ~/test_dir.txt` # `~` means “home directory” (shell expansion)

Essential symbols

Symbol	Name	Linux/Bash Shortcut	macOS Shortcut	Function
\	Backslash	Alt Gr + ?	Shift + 7 (DE)	Escape character, file paths
/	Forward Slash	Shift + 7	/ (DE -)	Directory separator, command options
`	Backtick	Right of the ? key	Option + `	Command substitution (e.g., `ls`)
	Pipe	Alt Gr + <	Shift + \ (on DE #)	Pipe output between commands
;	semicolon		;(DE = ö)	Multiple command separator (\$date ; ls)
[]	Brackets	Alt Gr + 8 or 9	[]	Test expressions
{ }	Braces / Curly Brackets	Alt Gr + 7 or 0	Shift + [Shift +]	Grouping, brace expansion
< >	Angle Brackets	Right of Left Shift key	Shift + , Shift + .	Redirection
~ ~	Tilde	Alt Gr + +	Option + n, then space	Home directory shortcut
##	Hash / Pound	Shift + 3	Shift + 3	Comment line
*	Asterisk / Star	Shift + *	Shift + 8	Wildcard character, multiply
!	Exclamation Mark (Bang)	Shift + 1	Shift + 1	History expansion
@	At Sign	Alt Gr + Q / Shift + 2	Shift + 2	Used in arrays, email
\$	Dollar Sign	Alt Gr + 4	Shift + 4	Variable access
%	Percent	Shift + 5	Shift + 5	Job control
^	Caret	Shift + 6	Shift + 6	Regex (start of line), CTRL combos
&	Ampersand	Shift + 6 or 7	Shift + 7	Run command in background
()	Parentheses	Shift + 8 or 9	Shift + 9 / 0	Subshell grouping

Example Navigating the Linux File System

- pwd What is my current directory?
- ls List directory files
- cd Change directory
- cd ~ Go to home directory in /user
- cd .. Move down one directory
- cd ./ Move up one directory (cd test/test2/)
- history List my previous commands
- Up arrow Cycle through previous commands
- ps List current processes
- kill End current process
- clear Clear the screen

(CLI).... Manipulating Files and Directories

- `ls -lh` # List files (long format, human-readable sizes)
- `mkdir folder` # Create directories
- `touch file1` # Create an empty file
- `cp file1 file 2` # Copy a file (-a source.txt backup/source.txt)
- `cp -a folder1 folder2` # Copy a directory
- `mv file 1 file2` # Move/rename
- `rm -i file.txt` # Remove a file (interactive prompt)

Advanced File Manipulation

- `find . -type f -name "*.txt"` # Find files by name
- `find . -type f -mtime -2` # Find files modified in the last 2 days
- `find . -type f -size +100M -print` # Find large files (>100MB)
- `rename 's/ /_/g' *.txt` # Batch rename: replace spaces with underscores
- `rename -n 's/ /_/g' -- *.txt` (preview before)
 - `brew install rename` (MAC) `sudo apt install rename` (ubuntu)
- `diff -u f1.txt f2.txt` # Compare two files
- `diff -ur dirA/ dirB/` # Compare directories recursively

Using **grep** for Searching specific patterns

- `grep "original" original.txt` # prints matching lines
- `grep -n "original" original.txt` # Show line number of the match
- `grep -i "original" original.txt` # Case-insensitive search
- `grep -w "original" original.txt` # Match the whole word only

Awk for pattern scanning and processing

- `awk '{print $1}' original.txt` # Print specific fields (columns)
- `awk '{print $1, $3}' test.csv` # Print specific fields (columns)
- `awk -F',' '{print $1, $3}' test.csv` # Use a custom field separator (CSV)
- `awk -F',' '{print $1, $6}' test.csv`
- `awk '$1 == "7"' test.csv` # Show only the lines where column match
- `awk -F',' '$6 == "PASS"' test.csv` # Print 1 (true) / 0 (false) for each line

Connecting to Servers and copying files

- ssh user@server.example.com # # Basic SSH login
- scp file.txt user@server.example.com:/home/user/
 - # Copy a file to a server (SCP)
- scp -r mydir/ user@server.example.com:/home/user/
 - # Copy a directory to a server (SCP recursive)

Piping Commands and Input/Output

- `ls -la > listing.txt` # Redirect stdout to a file (overwrite)
- `echo "new entry" >> listing.txt` # Append stdout to a file
- `cmd1 && cmd2 && cmd3` # Combine commands; stop if one fails
- `cmd1 ; cmd2` # Run next command regardless of success

Tips

- **File naming conventions**

- NEVER use spaces in file names or directories

% touch NGS 101.txt

% touch NGS_101.txt

- Choose a Text Editors: Advanced is preferred (hard but helpful)
- UNIX files are often compressed as ‘tarballs’ (.tar) or ‘gunzip’ (gz)
- % tar --xzf file1.tar % gzip -d file1.gz
- install a new tool that was published in an academic paper > make all
- make it run from anywhere copy it into your /bin folder > cp * /bin

Exercise

- Create a test folder
- Create dummy files
- Re-direct directory content to a .txt file
- grep one file
- Show content

- `chmod +x create_and_process.sh`